

Front-End Relatório – Projeto Coony

1. Resumo Executivo

- **Objetivo:** documentar todo o front-end do Coony, cobrindo arquitetura, componentes, páginas, assets, tutoriais operacionais e impacto técnico.
- **Stack:** Django Templates + vanilla JS + CSS modularizado em `usuarios/static/` e bundles específicos por feature. Fonts principais: Poppins, Montserrat, Roboto e Material Symbols.
- **Princípios:** componentização via `{% include %}`, responsividade controlada por CSS + `matchMedia`, feedback consistente usando toasts integrados ao `django.contrib.messages`.

2. Arquitetura de Pastas Front-End

Diretório	Papel	Observações
<code>usuarios/templates/usuarios/</code>	Páginas autenticadas (dashboard, social, eventos, etc.)	Cada template inicia com <code>{% load static %}</code> , inclui <code>toast.html</code> , <code>messages.html</code> e os shells de navegação desktop/mobile.
<code>usuarios/templates/pages/landing_page/</code>	Landing pública	<code>index.html</code> usa assets dedicados em <code>static/pages/landing_page/</code> .
<code>usuarios/templates/usuarios/components/</code>	Componentes reutilizáveis	<code>sidebar.html</code> , <code>navbar_mobile.html</code> , <code>toast.html</code> , <code>messages.html</code> , snippets auxiliares.
<code>usuarios/static/css/</code>	Estilos globais herdados	Ex.: <code>dashboard.css</code> , <code>perfil.css</code> , <code>notifications.css</code> .
<code>usuarios/static/pages/<feature>/</code>	Bundles isolados por página	Mantém dependências da landing, eventos, etc.
<code>usuarios/static/script/</code>	JS compartilhado (<code>app.js</code> , <code>chat.js</code> , <code>redirectscriptMobile.js</code>)	Interações gerais e WebSocket do chat.
<code>media/</code>	Uploads de usuários (logos, fotos, eventos)	Consumidos nos templates via <code>{{ model.image.url }}</code> .

3. Componentes Compartilhados

Sidebar (`usuarios/components/sidebar.html`)

- Renderiza navegação principal e blocos de ação (ex.: criar evento, empresa).
- Usa atributos `data-route/data-match` e um script inline para destacar a rota atual com base em `body.dataset.currentRoute`.
- Contém condicionais para permissões corporativas (`user.empresaprofile`).

Navbar Mobile (`usuarios/components/navbar_mobile.html`)

- Barra fixa inferior para telas $\leq 768\text{px}$.
- Herda o mesmo set de rotas da sidebar, exibindo avatar (ou `default-avatar.svg`) e ícones Material Symbols.

Sistema de Toast (`usuarios/components/toast.html` + `messages.html`)

- `toast.html` define markup base, CSS tokens (`--toast-success`, etc.) e API JS (`Toast.push({...})`).
- `messages.html` itera sobre `django.contrib.messages`, invoca `Toast.push` com os níveis de severidade corretos.
- Resultado: qualquer view que use `messages.success/error` dispara notificações automaticamente.

4. Páginas-Chave

Landing Page (`templates/pages/landing_page/index.html`)

- Seções: Hero, Inscrição, Equipes (Design/Marketing/Front/Back/Documentação), Equipe Pedagógica, Footer.
- CTAs principais (`Quero me inscrever`, `Ir para o Aplicativo`) direcionam para `{% url 'login' %}` conforme requisitado.
- Carrosséis de equipes usam estrutura `.carousel__items` com botões prev/next e imagens em `static/pages/landing_page/imagens-*`.

Dashboard (`templates/usuarios/dashboard.html`)

- Ponto inicial autenticado, combina cards de eventos, atalhos e feed.
- Inclui `sidebar.html`, `navbar_mobile.html`, `toast.html`, `messages.html`.
- JS: gerencia favoritos via fetch e manipula nav responsivo.

Social (`templates/usuarios/social.html`)

- Feed com postagens, curtidas e comentários.
- Alternância entre sidebar/nav mobile através de script inline idêntico ao do dashboard.
- Formulários de postagem e comentários usam `csrf_token` padrão.

Eventos (`templates/usuarios/eventos.html`)

- Filtros: busca, modalidade, dificuldade, visualização (lista/grade).
- Renderiza cards com `data-modalidade` e `data-nivel` para JS (`static/pages/eventos/js/eventos.js`) aplicar filtros instantâneos.
- Botão "Criar Evento" dispara `{% url 'create_event' %}`.

Favoritos (`templates/usuarios/favoritos.html`)

- Tema escuro dedicado (inline CSS no arquivo).
- Usa fetch `POST` para `toggle_favorite_event` com `csrftoken` e atualiza DOM sem recarregar.
- Mostra estado vazio com CTA para `eventos_list` quando não há favoritos.

Criação de Evento (`templates/usuarios/create_event.html`)

- Baseado no `form` Django, exibindo `form.field.errors` ao lado de cada input.
- Explica upload de imagens e oferece layout responsivo (nav desktop/mobile alternado via script).

Notificações (`templates/usuarios/notifications.html`)

- Hero com métricas (`stats.total`, `stats.like`, etc.) e "Atualizado em"/"Limpo há".
- Formulário GET para filtros (`q`, `type`), ações POST (`clear`).
- Feed renderiza `notification.type/icon/cta_url/relative_time` com badges e CTA "Ir".

Perfil (`templates/usuarios/perfil.html`)

- Integração com Cropper.js para corte client-side de avatar; manipula `File` via `DataTransfer` para reenviar blob cortado.
- Exibe selos (`profile-crown`) baseados em `user.gm_permission_level`.
- Modalidades tratadas como tags editáveis e persistidas em input hidden.

Chat (`templates/usuarios/chat.html`)

- Estrutura do chat + navs + toasts.
- JS principal em `static/script/chat.js` usa `dataset` para endpoints WebSocket/REST.

5. Assets & Build

- **CSS modular:** arquivos legacy (`css/dashboard.css`, `css/notifications.css`) convivem com bundles novos (`pages/eventos/css/eventos.css`). Recomendável centralizar tokens (cores, tipografia) em um único arquivo base.
- **JS:** scripts pequenos inline para nav responsive/ favoritos; scripts maiores ficam em `static/script/` ou `pages/.../js/` (ex.: `pages/eventos/js/eventos.js`). Não há bundler, permitindo deploy direto no Django.
- **Fonts/Ícones:** import via CDN (Google Fonts, Remixicon, Material Symbols). Avalie self-host para reduzir requisições externas se necessário.

6. Fluxos e Interações

1. **Login → Dashboard:** nav desktop (sidebar) carregado por padrão; `matchMedia` revela navbar mobile <768px. Toasts exibem mensagens de login.
2. **Criar Evento:** usuário acessa `create_event`, envia formulário multipart com imagens. Validações retornam erros inline + toast.
3. **Favoritar Evento:** em `dashboard/social/eventos`, botão favorite dispara fetch para `toggle_favorite_event`, atualiza ícone e badge. `favoritos.html` mostra estado consolidado.
4. **Chat:** `chat.js` abre WebSocket, injeta mensagens em timeline e utiliza toasts para feedback de entrega/erros.
5. **Notificações:** GET com filtros atualiza a lista; POST `clear` limpa feed, mostrando estado vazio com CTAs para Social/Chat.

7. Tutoriais Operacionais

7.1 Adicionar novo item ao menu

1. Atualize `usuarios/components/sidebar.html` adicionando `<a data-route="nova-rot" ...>`.
2. Repita o link em `navbar_mobile.html` para manter paridade.
3. No template alvo, defina `<body data-current-route="nova-rot">` para que o script de highlight funcione.

7.2 Criar página autenticada reutilizando componentes

1. Crie `usuarios/templates/usuarios/minha_pagina.html` com `{% load static %}`.
2. Inclua toasts/messages + shells:

```
{% include 'usuarios/components/toast.html' %}
{% include 'usuarios/components/messages.html' %}
<div id="nav-mobile" ...>{% include 'usuarios/components/navbar_mobile.html' %}
</div>
<div id="nav-desktop" ...>{% include 'usuarios/components/sidebar.html' %}</div>
```

3. Adicione o script `matchMedia` (ou mova para `static/script/app.js` e apenas chame).
4. Crie `usuarios/static/pages/minha_pagina/css/style.css` e importe em `<head>`.

7.3 Acoplar mensagens de backend ao sistema de toast

1. Na view Django: `messages.success(request, 'Perfil atualizado!')`.
2. Nenhuma mudança no template: `messages.html` já injeta `<script>Toast.push(...)</script>` conforme o nível (`success`, `error`, `warning`).

7.4 Estender filtros de eventos

1. No template `eventos.html`, adicione novo campo (ex.: `<select id="filtro-bairro">`).
2. Insira `data-bairro="{ { evento.bairro|lower } }"` nos cards.
3. Em `usuarios/static/pages/eventos/js/eventos.js`, dentro de `filtrarEventos()`, leia `const bairro = document.getElementById('filtro-bairro').value;` e compare com `card.dataset.bairro` para exibir/ocultar.

8. Impacto, Riscos e Recomendações

- **Consistência visual:** Landing possui estilo próprio; considere extrair tokens (cores/spacing) para um arquivo comum e importar tanto na landing quanto nas páginas internas.
- **JS duplicado:** lógica `matchMedia` para alternar nav é repetida em vários templates. Centralizar em `static/script/app.js` reduz divergência e facilita manutenção.
- **Inline CSS extenso** (`favoritos.html`, `sidebar.html`): mover para arquivos dedicados (`static/css/favoritos.css`, `static/css/sidebar.css`) melhora caching e evita colisões de estilo.
- **Toast API:** hoje cada template injeta `Toast.push` via mensagens; criar um wrapper JS exportado (`static/script/toast.js`) padroniza chamadas diretas e documenta os níveis disponíveis.
- **A11y:** componentes já usam `aria-label` e `sr-only`, mas carrosséis da landing não têm foco gerenciável via teclado. Avalie adicionar roles + listeners para setas.
- **Testes automatizados:** inexistentes para UI. Implementar smoke tests (Playwright) para fluxos críticos (login, criar evento, favoritar, chat) mitigaria regressões.

9. Próximas Ações Sugeridas

1. Criar `static/css/shell.css` consolidando estilos da sidebar/navbar e remover duplicações inline.
2. Mover scripts compartilhados (nav toggle, Toast helpers) para `static/script/app.js` e importar via `<script src="{% static 'script/app.js' %}"></script>`.
3. Documentar tokens de design (cores, tipografia, espaçamentos) num guia rápido dentro deste repositório ou em `TOAST_IMPLEMENTATION.md`.
4. Avaliar self-host de fontes para performance e disponibilidade offline.
5. Configurar testes end-to-end mínimos para garantir funcionamento das principais telas antes de deploys.